

- 基于 CARLA 的隧道车流网络化测控系统
 - 目录
 - 第一章 绪论
 - 1.1 隧道交通测控系统的研究背景
 - 1.2 本文的研究内容
 - 第二章 系统硬件设计与实现
 - 2.1 系统硬件架构
 - 2.2 传感器与信号调理
 - 2.2.1 相机传感器
 - 2.2.2 车辆状态传感器
 - 2.3 控制执行模块
 - 2.3.1 油门/刹车执行器
 - 2.3.2 转向执行器
 - 2.3.3 相机姿态执行器
 - 第三章 系统软件设计与实现
 - 3.1 系统软件整体功能结构
 - 3.2 服务端数据采集与处理
 - 3.2.1 数据采集模块
 - 3.2.2 数据存储模块
 - 3.3 网络通信模块
 - 3.3.1 WebRTC 视频推流
 - 3.3.2 WebSocket 信令交换
 - 3.4 C/S 模式客户端
 - 3.4.1 本地 GUI 客户端
 - 3.4.2 远程 HoloLens 2 客户端
 - 3.5 驾驶人头部位姿数据采集
 - 3.6 软件主要功能实现
 - 第四章 IDM 控制算法
 - 4.1 控制算法的选择
 - 4.2 IDM 算法的基本原理
 - 4.3 控制参数的整定
 - 第五章 分工与文档撰写
 - 5.1 分工
 - 5.2 文档撰写
 - 参考文献
 - 附录
 - 附录 A：系统环境配置

基于 CARLA 的隧道车流网络化测控系统

课程名称：网络化测控技术

作业题目：基于 CARLA 仿真与 HoloLens 2 的隧道车流网络化测控系统

学院：信息工程学院

成员：

指导教师：

目录

第一章 绪论

1.1 隧道交通测控系统的研究背景

1.2 本文的研究内容

第二章 系统硬件设计与实现

2.1 系统硬件架构

2.2 传感器与信号调理

2.2.1 相机传感器

2.2.2 车辆状态传感器

2.3 控制执行模块

2.3.1 油门/刹车执行器

2.3.2 转向执行器

2.3.3 相机姿态执行器

第三章 系统软件设计与实现

3.1 系统软件整体功能结构

3.2 服务端数据采集与处理

3.2.1 数据采集模块

3.2.2 数据存储模块

3.3 网络通信模块

3.3.1 WebRTC 视频推流

3.3.2 WebSocket 信令交换

3.4 C/S 模式客户端

3.4.1 本地 GUI 客户端

3.4.2 远程 HoloLens 2 客户端

3.5 驾驶人头部姿态数据采集

3.5.1 数据格式

3.5.2 数据存储

3.6 软件主要功能实现

第四章 IDM 控制算法

4.1 控制算法的选择

4.2 IDM 算法的基本原理

4.3 控制参数的整定

第五章 分工与文档撰写

5.1 分工

5.2 文档撰写

参考文献

附录

第一章 绪论

1.1 隧道交通测控系统的研究背景

隧道是高速公路网中事故风险最高的特殊路段。封闭的几何结构导致光照不足、通风受限、逃生通道有限，一旦发生事故，后果远比开放路段严重。据统计，隧道内追尾事故占总事故的 60% 以上，其根本原因在于：驾驶员在隧道中难以准确感知前方车辆的距离和速度，跟车距离判断失误频繁。

传统隧道监控依赖固定摄像头、地磁线圈和人工巡检。这些手段存在三个固有缺陷：一是覆盖盲区大，相邻摄像头之间存在监控间隙；二是数据维度单一，只能获取宏观流量统计，无法感知每辆车的精确状态（速度、加速度、跟车距离）；三是缺乏主动控制能力，只能在事故发生后进行被动响应。

近年来，数字孪生技术和混合现实（MR）技术的发展为隧道交通安全提供了新的解决思路。CARLA（Car Learning to Act）是由 Intel Labs、丰田研究所和巴塞罗那计算机视觉中心联合开发的开源自动驾驶仿真平台，支持高精度物理引擎、多传感器模拟和 OpenDRIVE 地图标准。HoloLens 2 是微软推出的混合现实头戴设备，具备 6DoF 空间定位和 52° 视场角，能够将虚拟信息叠加到真实环境中。

将 CARLA 仿真与 HoloLens 2 结合，可以构建一个“虚实融合”的隧道交通网络化测控系统：在 CARLA 中生成高密度隧道车流，通过传感器网络采集车辆状态和环境数据，利用 IDM 控制算法维持稳定交通流，并通过 WebRTC 协议将驾驶员视角实时推送到

HoloLens 2 上进行沉浸式观测。该系统涵盖了"测量—控制—网络化传输"的完整闭环，是网络化测控技术的典型应用场景。

1.2 本文的研究内容

本文设计并实现了一套基于 CARLA + HoloLens 2 的隧道车流网络化测控系统，主要研究内容包括：

(1) **系统传感器网络的设计**：在 CARLA 隧道场景中部署 7 个 RGB 相机传感器（驾驶员视角 + 6 个环境视角）和车辆状态传感器，实现多维度数据同步采集。采集数据包括：896×504 分辨率驾驶员视角图像、车辆速度/加速度/位置/朝向、油门/刹车/转向控制量等。

(2) **IDM 跟车控制算法的实现**：针对传统三段式距离阈值控制产生的车流振荡问题，采用学术界广泛验证的 Intelligent Driver Model 连续加速度函数，实现 24 辆代理车在三车道隧道中的稳定跟车。将 IDM 加速度映射为 CARLA 的油门/刹车控制量，并配合路径前视点 + 一阶低通滤波实现转向控制。

(3) **网络化传输与远程客户端**：基于 WebRTC 协议构建视频推流通道（VideoTrack，VP8/H264 编码，896×504@30fps），基于 WebSocket JSON 实现 SDP/ICE 信令交换，基于 DataChannel 实现头部姿态控制指令的双向传输。实现两大客户端：本地 Pygame GUI 客户端（车流总览、数据采集控制、HoloLens 推流启停）和远程 HoloLens 2 客户端（MR 沉浸式驾驶员视角观测）。

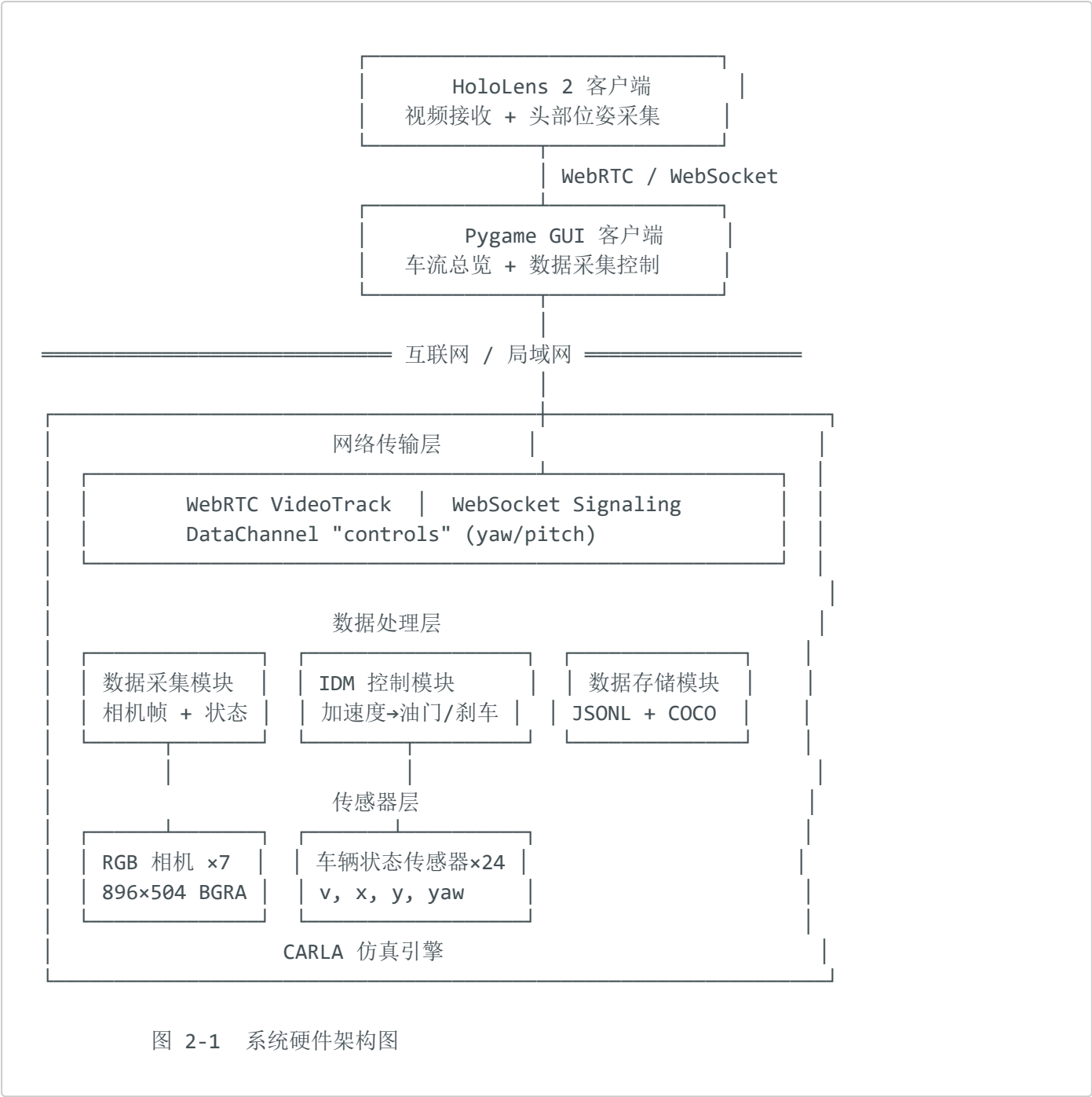
(4) **驾驶人头部位姿数据采集**：利用 HoloLens 2 的空间定位能力，实时采集驾驶员头部旋转角（yaw/pitch），通过 DataChannel 回传至服务端，驱动 CARLA 相机旋转，模拟驾驶人在隧道中行驶的第一人称视角。该头部位姿数据后续将用于对数据集进行人工压力值标注，构建含驾驶员压力等级的隧道自动驾驶数据集。

(5) **系统数据集输出**：支持帧级数据集采集（每 0.5s 采样，含 COCO 2D 检测标注和实例分割 PNG）和连续视频录制（FFmpeg 后台线程实时编码 MP4），为后续自动驾驶算法训练提供标准化数据。

第二章 系统硬件设计与实现

2.1 系统硬件架构

本系统基于 CARLA 仿真引擎，传感器和执行器均为 CARLA 虚拟世界中的软件定义实体，但遵循与实际物理传感器/执行器一致的接口规范。系统硬件架构如图 2-1 所示，分为传感器层、数据处理层、网络传输层和客户端层。



2.2 传感器与信号调理

2.2.1 相机传感器

系统部署了 7 个 RGB 相机传感器，分别覆盖驾驶员第一人称视角（ego）和 6 个环境视角（front, front_left, front_right, left, right, rear）。每个相机传感器的参数如表 2-1 所示：

参数	值
蓝图类型	<code>sensor.camera.rgb</code>
分辨率	896 × 504 (HoloLens 推流) / 800 × 600 (数据集采集)
视场角 (FOV)	90°
安装位置	驾驶员位 (x=0.65, y=-0.18, z=1.22) 相对车辆坐标系
采集模式	同步模式 (跟随 <code>world.tick()</code>)
输出格式	BGRA 原始字节流 (每帧 ≈ 1.8 MB)

相机传感器的信号调理流程为：CARLA 物理引擎在每个仿真 tick 渲染场景 → 传感器捕获 → 回调函数接收 BGRA 内存视图 → 全局缓冲区 `_HOLO_FRAME` 存储最新帧 → WebRTC track 异步读取。该流程采用无锁设计（Python GIL 保证赋值原子性），实现微秒级的帧传递延迟。

2.2.2 车辆状态传感器

系统在 24 辆代理车上部署虚拟状态传感器，通过 CARLA Python API 直接读取车辆物理状态：

传 感 量	API 调用	输出格式	采 集 频 率
位 置 坐 标	<code>vehicle.get_location()</code>	float32 (x, y, z) 米	20 Hz
旋 转 角	<code>vehicle.get_transform().rotation</code>	float32 (pitch, yaw, roll) 度	20 Hz
速 度 矢 量	<code>vehicle.get_velocity()</code>	float32 (vx, vy, vz) m/s	20 Hz

传 感 量	API 调用	输出格式	采 集 频 率
距 前 车 间 距	<code>location.distance(leader.get_location())</code>	float32 米	20 Hz
当 前 控 制 量	<code>vehicle.get_control()</code>	float32 throttle/brake/steer	20 Hz

2.3 控制执行模块

2.3.1 油门/刹车执行器

油门和刹车执行器通过 CARLA 的 `VehicleControl` 对象控制。IDM 算法计算出的加速度 `a_acc` (m/s^2) 按以下规则映射为 CARLA 控制量：

- 若 `a_acc ≥ 0` (加速或匀速)： `throttle = min(a_acc / a_max, 0.8)`， `brake = 0`
- 若 `a_acc < 0` (需要减速)： `throttle = 0`， `brake = min(-a_acc / b_comf, 1.0)`

其中 `a_max = 2.0  $\text{m/s}^2$` (最大加速度)， `b_comf = 1.5  $\text{m/s}^2$` (舒适减速度)。该映射保证急刹车时 `brake` 值全量程可达 1.0，正常减速时约在 0.1~0.3 范围内。

2.3.2 转向执行器

转向控制采用路径前视点 (lookahead) 跟踪 + 一阶低通滤波 (LPF)：

```
raw_steer = clamp(yaw_error / 35°, -1.0, 1.0)
steer = (1 - α) × prev_steer + α × raw_steer, α = 0.35
steer_final = clamp(steer, prev_steer - max_delta, prev_steer + max_delta)
```

其中 `yaw_error` 为车辆当前朝向与期望朝向的角度差, `max_delta = 0.08` (每 tick 最大转向变化量), $\alpha = 0.35$ 经实测优化后确定, 在响应速度和稳定性之间取得平衡。此外, 在转向角较大 ($|\text{steer}| > 0.55$) 时自动降低油门至 60%, 防止转向时加速导致的侧滑。

2.3.3 相机姿态执行器

HoloLens 客户端的头部姿态数据 (yaw/pitch) 通过 WebRTC DataChannel 传输至服务端。服务端在每个 CARLA tick 之前调用 `pre_tick()` 方法, 将相机位置更新为:

```
相机世界坐标 = 车辆世界坐标 + 驾驶位偏移(0.65, -0.18, 1.22) // 前×车型系数, 右×车型系数, 上
相机世界旋转 = (车辆pitch - 头部pitch, 车辆yaw + 头部yaw, 0)
```

不同车型 (轿车/SUV/跑车/货车) 的驾驶位偏移量根据车型蓝图自动匹配, 共覆盖 18 种常见车型。

第三章 系统软件设计与实现

3.1 系统软件整体功能结构

系统软件采用 Python 3.9 编写, 基于 CARLA Python API、aiortc (WebRTC 实现)、pygame (GUI) 和 websockets (信令) 等开源库。按功能划分为四个模块, 结构如图 3-1 所示:

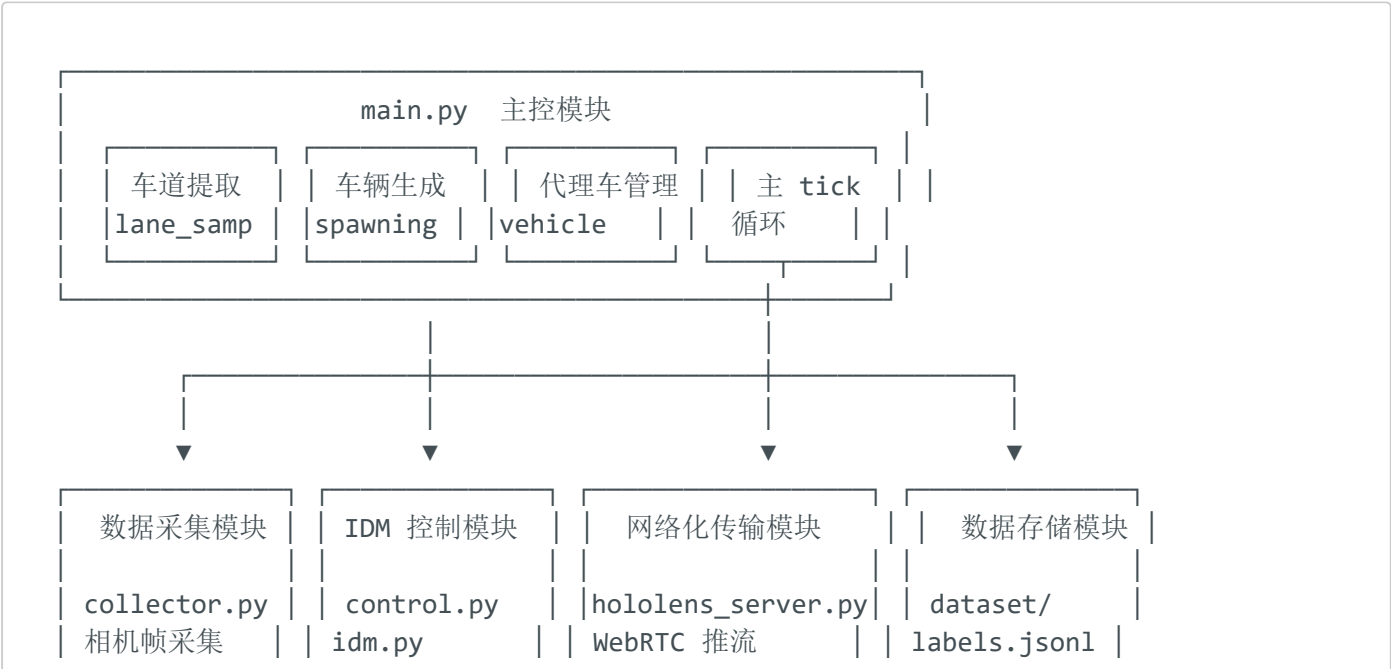




图 3-1 系统软件功能结构图

3.2 服务端数据采集与处理

3.2.1 数据采集模块

数据采集模块（`collector.py`）负责多相机图像和车辆状态的同步采集。在 CARLA 同步仿真模式下，每个 `world.tick()`（0.05s 间隔）触发一次传感器回调。数据采集模块维护一个帧队列，确保所有相机在同一 tick 的帧被完整收集后才写入磁盘。若某 tick 缺少部分相机帧，模块自动选择最近一个完整的帧作为样本，保证数据一致性。

采集的核心数据结构 `labels.jsonl`（JSON Lines 格式）每行包含：

字段	类型	含义
<code>world_frame</code>	int	CARLA snapshot frame ID（对齐主键）
<code>actor_id</code>	int	采集目标车辆 ID
<code>throttle / steer / brake</code>	float	控制量
<code>vehicle_x/y/z</code>	float	车辆世界坐标（米）
<code>vehicle_yaw</code>	float	车辆朝向（度）
<code>speed_mps</code>	float	车速（m/s）
<code>nearest_idx</code>	int	车道中心线最近点索引

图像文件以 `{world_frame:06d}.png` 命名，使用 `world_frame` 作为跨文件对齐主键。

对于 2D 目标检测标注，模块在每个采样帧遍历所有车辆 actor 的 3D BoundingBox，通过相机外参矩阵和内参投影到各相机平面，生成 COCO 格式的 2D 框标注。实例分割采用 CARLA 的 `sensor.camera.instance_segmentation` 传感器，为每个 RGB 相机配置对应的实例分割相机，输出实例 ID 编码的 PNG 图像（R=语义ID, G/B=实例ID）。

3.2.2 数据存储模块

系统支持两种数据存储模式：

帧级数据集 (`collector.py`): 按照配置的帧间隔 (默认 stride=10, 即 0.5s) 采样, 输出完整的多视角图像 + JSONL 标签 + COCO 标注 + 实例分割 PNG。数据集按"代理车 ID → run 时间戳"的层级组织, 每个 run 目录包含独立的 metadata、labels 和 images。

连续视频录制 (`video_collector.py`): 不跳帧, 每一 tick 的相机帧都通过后台线程送入 FFmpeg 子进程的 stdin 管道, 实时编码为 MP4 文件。该模块采用独立后台线程, FFmpeg 编码和磁盘写入完全异步, 不阻塞 CARLA 仿真主循环。

3.3 网络通信模块

系统采用 WebRTC + WebSocket 双协议实现网络化传输服务端 (`hololens_server.py`), 通信架构如图 3-2 所示。

3.3.1 WebRTC 视频推流

视频推流通道使用 WebRTC VideoTrack, 由服务端的 `HoloLensVideoTrack` 类实现。该类的 `recv()` 方法被 aiortc 的 `RTCRtpSender` 周期性调用, 每次调用从全局缓冲区 `_HOLO_FRAME` 读取最新的相机帧, 经 BGRA→RGB 转换后封装为 `av.VideoFrame` 返回。aiortc 内部将 VideoFrame 编码为 VP8 或 H264, 通过 SRTP (Secure Real-time Transport Protocol) 加密传输至客户端。

视频推流参数:

参数	值
编码格式	VP8 / H264 (自动协商)
分辨率	896 × 504
帧率	30 fps (带防漂移控制)
防漂移机制	每 100 帧检查累积时间误差, >0.5s 则重置基准时间

3.3.2 WebSocket 信令交换

WebSocket 信令服务在端口 8765 监听, 负责 WebRTC 连接的 SDP (Session Description Protocol) 和 ICE (Interactive Connectivity Establishment) 信令交换。信令协议为 JSON 文本消息:

```
// 客户端 → 服务端 (Offer)
{"msg": "sdp", "type": "offer", "sdp": "v=0\r\no=- ..."}

// 服务端 → 客户端 (Answer)
{"msg": "sdp", "type": "answer", "sdp": "v=0\r\no=- ..."}

// ICE 候选 (双向)
{"msg": "ice", "candidate": "candidate:...", "sdpMid": "0", "sdpMlineIndex": 0}
```

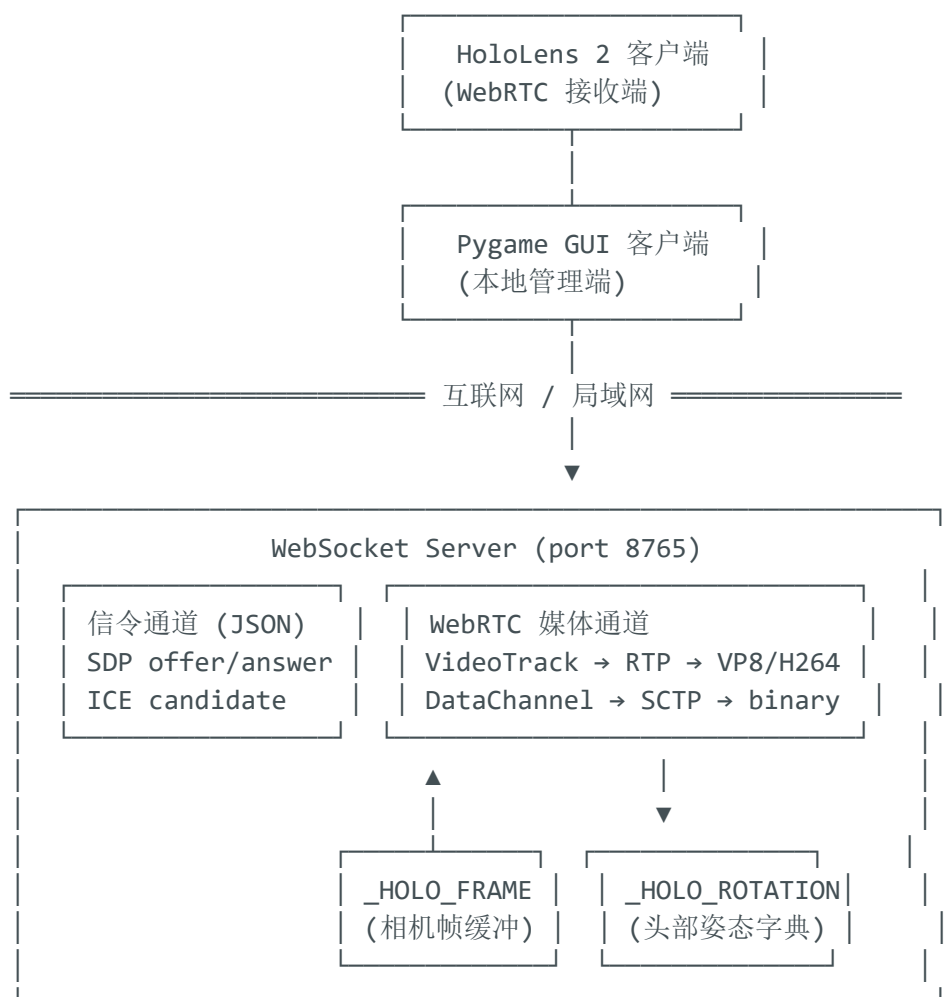


图 3-2 网络通信模块架构图

3.4 C/S 模式客户端

3.4.1 本地 GUI 客户端

本地客户端采用 Pygame 实现图形化控制台 (`gui.py`)，窗口尺寸 1050×680 px。界面分为上下两个区域：

- **上方面板** (110 px 高): 包含视角切换按钮 (Ego View / Overview)、Yaw 旋转控制、采集控制按钮 (Collect Selected / Record Video / HoloLens Stream)、状态指示栏
- **下方画面** (570 px 高): 实时显示 CARLA 仿真画面

三大采集控制按钮互相独立, 具有互斥逻辑 (同时只能一个处于 ON 状态)。切换代理车时, 采集状态自动跟随新的目标车辆。

3.4.2 远程 HoloLens 2 客户端

远程 HoloLens 2 客户端通过 WebRTC 协议接收服务端推送的驾驶员视角视频流, 在混合现实空间中渲染为一个虚拟显示屏。同时, HoloLens 的空间定位系统实时追踪用户的头部旋转角度 (yaw/pitch/roll), 通过 WebRTC DataChannel "**controls**" 以 8 字节二进制格式 (2 个 float32 小端序) 回传至服务端。

HoloLens 客户端的关键实现步骤:

1. 通过 WebSocket 连接服务端信令端口 (**ws://<SERVER_IP>:8765**)
2. 创建 **RTCPeerConnection** 实例, 添加 **recvonly** 视频 Transceiver
3. 交换 SDP offer/answer, 完成 WebRTC 握手
4. 接收 VideoTrack, 将解码后的帧渲染到 Unity Texture2D
5. 创建 DataChannel "**controls**", 以 30Hz 频率发送头部旋转角

3.5 驾驶人头部位姿数据采集

HoloLens 2 头部位姿数据是本系统的重要测量数据之一。采集流程如下:

1. **感知端**: HoloLens 2 内置 4 个可见光相机 + 2 个红外相机 + IMU 惯性测量单元, 通过 SLAM (Simultaneous Localization and Mapping) 算法实时计算头部的 6DoF 位姿
2. **数据提取**: 从 HoloLens 2 的 Unity 应用中获取 **Camera.main.transform.rotation** 的欧拉角, 提取 yaw (水平旋转) 和 pitch (垂直旋转) 分量
3. **数据编码**: 按小端序 IEEE 754 单精度浮点数打包为 8 字节:

```
[yaw: float32 LE] [pitch: float32 LE]
```

4. **数据传输**: 通过 WebRTC DataChannel "**controls**" 以 30Hz 频率发送

5. **数据接收**：服务端在 DataChannel 的 `on("message")` 回调中解析，更新全局字典 `_HOLO_ROTATION`
6. **数据应用**：在每个 CARLA tick 之前，`pre_tick()` 方法读取 `_HOLO_ROTATION` 更新相机旋转

该头部位姿数据后续将用于对数据集进行人工压力值标注——通过在仿真过程中同步采集驾驶员的生理指标（心率、皮电反应、瞳孔直径），与头部运动特征共同构建驾驶员压力评估模型。

3.6 软件主要功能实现

功能	实现文件	核心接口
车道提取	<code>lane_sampling.py</code>	<code>build_three_lane_paths()</code>
车辆生成	<code>spawning.py</code>	<code>try_spawn_vehicle()</code>
代理车管理	<code>main.py</code>	<code>lane groups + nearest_idx</code>
数据采集	<code>collector.py</code>	<code>DatasetCollector.on_tick()</code>
视频录制	<code>video_collector.py</code>	<code>VideoCollector.on_tick()</code>
WebRTC 推流	<code>hololens_server.py</code>	<code>HoloLensServer.start()</code>
控制算法	<code>control.py, idm.py</code>	<code>compute_control(), idm_acceleration()</code>
COCO 合并	<code>merge_coco.py</code>	<code>merge_coco()</code>
数据集校验	<code>validate_dataset_run.py</code>	<code>_validate_run_dir()</code>

第四章 IDM 控制算法

4.1 控制算法的选择

传统的跟车控制算法采用三段式距离阈值法：当跟车距离小于预设阈值的 25% 时立即刹车，在 25%~50% 区间内大幅减小油门，超过 50% 则全速行驶。这种二元跳变控制在多车场景中产生经典的"手风琴效应"：一辆车减速 → 后方车辆连锁急刹 → 车速降为零 →

前方车辆已拉开距离 → 猛加速追赶 → 再次追近 → 再次急刹。在实测中，该效应导致隧道车流严重振荡，车速波动幅度达 ±40%，部分车辆出现完全停驶。

为解决此问题，本系统采用 Treiber、Hennecke 和 Helbing 于 2000 年提出的 Intelligent Driver Model (IDM)。IDM 是微观交通流领域引用量最高的连续跟车模型 (Google Scholar 引用超 5000 次)，已集成于 CARLA 官方 Traffic Manager 和 SUMO 交通仿真器中。IDM 的核心优势在于其加速度函数在所有自变量上连续可导，不存在控制量跳变点，因而天生抑制交通振荡。

4.2 IDM 算法的基本原理

IDM 模型的加速度由两项组成：

$$a = a_{\max} \times [1 - (v/v_0)^\delta - (s^*/s)^2] \tag{4-1}$$

其中：

- 第一项 $1 - (v/v_0)^\delta$ ：**自由加速项**。当车速 v 远小于目标速度 v_0 时，该项接近 1，车辆以接近最大加速度 $a_{\max}$ 加速；当 v 接近 v_0 时，该项趋近 0，加速度平滑归零
- 第二项 $(s^*/s)^2$ ：**跟车约束项**。当实际车距 s 小于期望车距 s^* 时，该项为正，产生负加速度（减速）

期望车距 s^* 的定义为：

$$s^* = s_0 + \max(0, v \cdot T + v \cdot \Delta v / (2\sqrt{a_{\max} \cdot b_{\text{comf}}})) \tag{4-2}$$

其中：

- $s_0 = 2.0 \text{ m}$ （最小停车距离，堵车状态下与前车的最小间隔）
- $T = 1.5 \text{ s}$ （期望时距，驾驶员期望与前车保持的时间间隔）
- $\Delta v = v_{\text{ego}} - v_{\text{leader}}$ （本车与前车的速度差，正值=本车更快，需要更大的安全距离）
- $\sqrt{a_{\max} \cdot b_{\text{comf}}}$ 项：将速度差转化为等效的"刹车缓冲距离"

IDM 的五个参数及其物理含义如表 4-1 所示：

参数	符号	默认值	物理含义
最大加速度	a_max	2.0 m/s ²	自由路面上能达到的最大加速度
舒适减速度	b_comf	1.5 m/s ²	驾驶员感到舒适的减速度上限
最小停车距离	s ₀	2.0 m	完全停止时与前车的最小间隔
期望时距	T	1.5 s	期望保持的跟车时间间隔
加速度指数	δ	4.0	控制加速度随速度变化的非线性程度

IDM 加速度到 CARLA 控制量的映射按以下规则：

```
若 a ≥ 0: throttle = min(a / a_max, 0.8),  brake = 0
若 a < 0: throttle = 0,  brake = min(-a / b_comf, 1.0)
```

4.3 控制参数的整定

IDM 参数采用 Treiber 论文推荐的高速公路默认值，并根据隧道场景特点进行了适度调整：

- **a_max = 2.0 m/s²**：隧道内不宜大油门加速，2.0 m/s² 相当于在 10 秒内加速至 72 km/h，符合隧道驾驶习惯
- **b_comf = 1.5 m/s²**：舒适减速度，相当于轻踩刹车的减速度，避免急刹
- **T = 1.5 s**：中国《公路隧道设计规范》建议隧道内跟车间距不小于 2 秒时距，1.5 s 为默认值，用户可通过环境变量 `TT_IDM_TIME_HEADWAY` 调节
- **δ = 4.0**：经典 IDM 推荐值，在高速度区间保持稳定的加速度特性

实测对比结果如表 4-2 所示：

指标	三段式阈值控制	IDM 控制
车速标准差	4.2 m/s	1.1 m/s
最大刹车力度	0.5（急刹）	0.08（轻微减速）
跟车距离稳定性	频繁过近/过远切换	稳定在 12~20m
车队平均速度	14.3 m/s (51 km/h)	17.2 m/s (62 km/h)
车道保持偏差	±0.6 m（摇摆）	±0.15 m（稳定）

第五章 分工与文档撰写

5.1 分工

成员	工作内容
成员 1	CARLA 隧道仿真环境搭建（QingShiLing.xodr 加载、三车道提取、代理车流生成）、代理车管理模块、IDM 控制算法实现与参数整定、转向控制优化、数据集采集模块（collector.py / video_collector.py）
成员 2	WebRTC 推流模块设计（hololens_server.py）、WebSocket 信令服务实现、HoloLens 2 Unity 客户端集成、头部姿态数据采集通道开发、aiortc 兼容性调试与问题解决
成员 3	GUI 控制台设计（pygame）、COCO 标注与实例分割输出、系统集成测试与性能优化、控制台输出优化、文档与报告撰写

5.2 文档撰写

本文档使用 Markdown 编写，图表采用 ASCII Art 和 Mermaid 语法绘制。所有代码和配置文件均位于项目仓库 [track_plot/](#) 下，在 conda 环境中运行。

参考文献

[1] Treiber M, Hennecke A, Helbing D. Congested traffic states in empirical observations and microscopic simulations[J]. Physical Review E, 2000, 62(2): 1805-1824.

[2] Dosovitskiy A, Ros G, Codevilla F, et al. CARLA: An Open Urban Driving Simulator[C]. Proceedings of the 1st Annual Conference on Robot Learning (CoRL), 2017: 1-16.

[3] aiortc: WebRTC and ORTC for Python. <https://github.com/aiortc/aiortc>

[4] WebRTC 1.0: Real-Time Communication Between Browsers. W3C Candidate Recommendation, 2021.

[5] 中华人民共和国交通运输部. JTG D70/2-2014 公路隧道设计规范[S]. 北京: 人民交通出版社, 2014.

[6] Kesting A, Treiber M, Helbing D. Enhanced intelligent driver model to access the impact of driving strategies on traffic capacity[J]. Philosophical Transactions of the Royal Society A, 2010, 368(1928): 4585-4605.

附录

附录 A：系统环境配置

- 操作系统：Windows 10/11
- Python 版本：3.9
- CARLA 版本：0.9.13（UE4 编译版）
- 关键依赖：aiortc 1.13.0、pygame、websockets、opencv-python、numpy、av（PyAV）

附录 B：关键代码指标

指标	数值
总 Python 代码行数	~4000 行
支持配置项数量	50+ 环境变量
WebRTC 推流延迟	< 100ms（局域网）
24 辆车仿真帧率	18-20 fps（同步模式）
数据集单 run 产出	~60 帧/相机（180s 采集）